Agentic Project Management

An enhancement of the emerging paradigm of
Spec-Driven Development

Research Study Participation Guide

Sub-Study B: GitHub Copilot



Conducted by:
Konstantinos Chaidemenos - CobuterMan

Institution:
National and Kapodistrian University of Athens



November 2025

1. Welcome & Introduction

Thank you for participating in this research study. Your participation is essential for evaluating how effectively modern AI-powered development tools support Spec-Driven Development (SDD) workflows.

The goal of this study is to compare how different AI-assisted frameworks perform on a practical, well-defined coding task. Please be assured that **you are not being tested**; the AI tools are. All data, including incomplete or non-functional code, is valuable to this research.

Data Privacy & Anonymity

All data that you will submit will be handled with strict confidentiality. Your name or any personal identifiers will not be linked to the data. The research artifacts you will provide will be used exclusively for this academic research and will be anonymized in any resulting publications. By proceeding, you consent to the anonymous collection and use of this data as described.

2. Study Overview

2.1. Your Task

Your task is to research an existing legacy PHP codebase and implement a new feature in an existing Python codebase based on your findings. You will receive a detailed **Product Requirements Document (PRD)** as the sole source of truth for the specification, guiding both you and the AI tools.

2.2. Your Tools

You will complete this task **twice**, each time using a different AI-coding framework (e.g., APM, Spec-kit, Copilot's Plan agent). Sessions will be **time-boxed and isolated on separate branches**. Between sessions, you will submit your code changes, chat logs, and a brief survey.

> **Learning Effects Note**
>
> We acknowledge that performing the task twice may introduce learning effects. To mitigate this, some participants will begin with Tool A, others with Tool B. This counterbalancing aims to ensure fair assessments across the study.

2.3. Your Time

Each session is strictly time-boxed to **three (3) hours**. It is important that you stop at the 3-hour mark, regardless of whether you are finished. Committing unfinished work is a perfectly valid and expected outcome.

2.4. Your Environment

To ensure the two sessions are completed in a controlled environment, set up your environment according to the following specifications:

- **Operating System:** POSIX-compliant systems such as Linux (e.g., Ubuntu 22.04+) or macOS (latest versions). For Windows, we recommend using Windows Subsystem for Linux (WSL) with a recent Ubuntu distribution.
- **APM v0.5.0+:** Make sure you have the latest version of APM installed for one of your sessions.
- **Workspace:** A dedicated directory with the following repositories cloned:
  - `eclass-mcp-server` (the main codebase where you will implement the new feature). Install dependencies using: `cd eclass-mcp-server && uv sync -dev -all-extras`
  - `openeclass` (the legacy PHP codebase where you will conduct research)
- **Python Version: 3.10+**
- **IDE: Visual Studio Code** of at least version 1.106
- **AI Tool: GitHub Copilot**

> **Important Copilot Access Note**
>
> This study requires access to GitHub Copilot's premium features. Note that **verified students are eligible for the GitHub Education Pack**, which provides free access to GitHub Copilot Pro. You can sign up for the GitHub Education Pack for free at: `https://education.github.com/pack`
>
> Please ensure you have access to GitHub Copilot's premium features in VS Code before beginning the session.

2.5. What You Will Deliver

At the end of each session, you will submit:

- A **code patch file** containing all changes made during the session. For example, a patch file `tool_a_solution.patch` for Tool A and `tool_b_solution.patch` for Tool B. See Section 4.2 for instructions on creating the patch file.
- The complete **AI interaction logs**, including exported chat conversations and Copilot request usage from your sessions. For example `tool_a_logs.zip` and `tool_b_logs.zip`. See Section 5 for exporting AI interaction logs from VS Code (Copilot).
- A brief **post-session survey** about your experience using each AI-coding framework. See Sections 4.2 and 4.3 for links to the surveys.

3. AI Tools: Setup & Instructions

Before starting your sessions, familiarize yourself with your assigned tools. Below are setup instructions and usage guidelines for each tool.

3.1. APM in VS Code (Copilot)

Description:

Agentic Project Management (APM) is designed to enhance your development process by decomposing specifications into actionable tasks and coordinating their execution across multiple AI agents; aiming to improve efficiency and context management.

Setup Instructions:

- Ensure you have `npm` installed on your system.
- Install the `agentic-pm` package globally using the following command:

```
npm install -g agentic-pm
```

- Once installed, use the `apm init` command within your working directory's root (alongside the other codebases) and select `GitHub Copilot`.

Video Tutorial & Familiarization:

To get acquainted with APM watch this tutorial and/or read the official documentation before starting your timed session.

3.2. Spec-kit in VS Code (Copilot)

Description:

Spec-kit is another AI-powered development framework designed to enhance the specification and planning stages of software development, ensuring a spec-first approach throughout the implementation process.

Setup Instructions:

- Ensure `uv` has been installed and set up on your system.
- Install the `specify-cli` using their official recommended installation command:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

- Once installed, use the `specify init <your-project-name>` command within your working directory's root (alongside the other codebases) and select `GitHub Copilot`.

Video Tutorial & Familiarization:

To get acquainted with Spec-kit watch this tutorial and/or read the official documentation before starting your timed session.

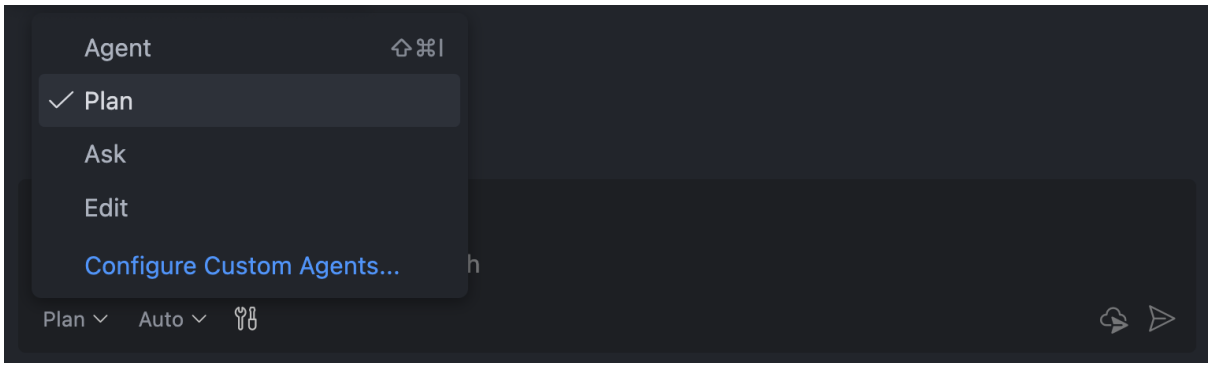
3.3. Plan agent in Copilot

Description:

Copilot's **Plan agent** implements Spec-Driven Development principles by emphasizing a "first-plan-then-execute" workflow. The agent is designed to research a task, break it down into manageable steps, ask clarifying questions, and present a full implementation plan **before** any code is written.

Setup Instructions:

No additional setup is required beyond having VS Code with GitHub Copilot installed. To enable the Plan agent, select it from the agent picker of Copilot's Chat panel:



Video Tutorial & Familiarization:

The Plan agent is integrated directly into Copilot. For a brief overview, see Copilot's official documentation on the Plan agent.

> **Important Familiarization Note**
>
> Before starting your timed session, it is **crucial** to familiarize yourself with the features and workflows of your assigned tool by watching the provided tutorials and reviewing the relevant documentation. You are encouraged to practice using the tool on small, unrelated tasks to build confidence. However, **please avoid using the actual task or codebase during practice to minimize learning effects.**
>
> **This preparation will help you use the tool effectively during the task.** In the post-session survey, you will be asked about your experience and familiarity with the tool.

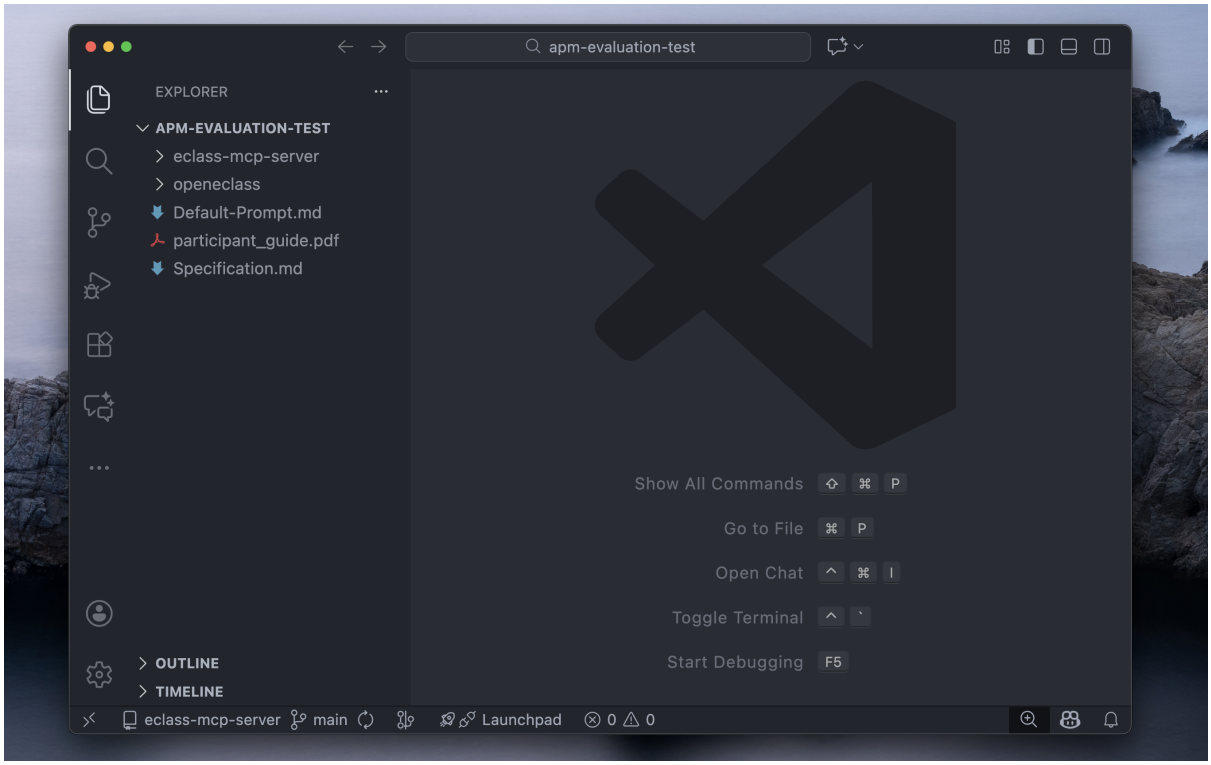
4. Session Workflow (Step-by-Step)

This section explains the session workflow in which you are to attempt to complete your task. Please follow these steps in order.

4.1. Ensure Environment is Set Up

Following Section 2.4, ensure you have installed VS Code with GitHub Copilot and that you have correctly set up your environment with all dependencies installed and the required repositories. You may include the `Specification.md` and the `Default_Prompt.md` files alongside this guide in your workspace for your convenience.

Opening VS Code in your designated workspace directory would look like this:



Verify that all the base dependencies are installed by running:

```
/PATH/TO/YOUR/PYTHON3 --version   # Check Python version
uv --version                      # Check UV version
npm --version                     # Check NPM version (for APM)

# Check Python codebase dependencies (silent success, errors shown)
cd eclass-mcp-server
/PATH/TO/YOUR/PYTHON3 -c "import mcp"
/PATH/TO/YOUR/PYTHON3 -c "import requests"
/PATH/TO/YOUR/PYTHON3 -c "import bs4"
/PATH/TO/YOUR/PYTHON3 -c "import dotenv"
```

4.2. Session 1: Tool A

Initialize Git and Create Branch:

Ensure you're starting from a clean state and create an isolated branch:

```
cd eclass-mcp-server                # go to implementation codebase root
git checkout main                   # switch to main branch
git checkout -b tool-a-attempt      # create new branch for Tool A patch
```

Before you start working:

- Launch your first assigned AI tool (Tool A) following its setup instructions in Section 3.
- **Thoroughly read the** `Specification.md` (the PRD) provided alongside this guide.
- Use the **Default Prompt** provided alongside this guide to initiate your conversation with your AI tool.
- Take a screenshot of the Copilot **Premium Requests usage bar** from your GitHub Copilot Settings page; this will serve as the **start-of-session** usage baseline.
- Use **only** your assigned Tool A for AI assistance during this session.
- Start the timer and get to work!

> **Important Copilot Workflow Note**
>
> Due to Copilot's request-based billing, participants should follow a **Hybrid Model strategy** when using these tools. This is to conserve your request quota.
>
> – **APM sessions:**
>
> * **Setup Phase:** Use **Claude Sonnet 4** for the initial **Setup Agent** session.
> * **Task Loop Phase:** Keep using **Claude Sonnet 4** for the **Manager Agent** and switch to whichever non-premium model you prefer for the **Implementation Agents** throughout the task execution. Non-premium models are the ones with a 0x multiplier in the model picker.
>
> – **Plan Agent and Spec-kit sessions:** use **Claude Sonnet 4** for planning and non-premium for execution.

Work Until Completion or Time Limit:

- Iterate on the task for **up to 3 hours**.
- **Stop at the 3-hour mark**, even if you haven't completed the task.
- Alternatively, stop when you have fully completed the task **(if before 3 hours)**.

Create Code Patch:

After the 3 hour mark or upon task completion, generate your solution patch file for this session:

```
git add . && git commit -m "Tool A solution"            # commit all changes
git diff main tool-a-attempt > tool_a_solution.patch    # create patch file
```

Git Note

Feel free to use Git for version control during your session. However, ensure that you **do not modify the main branch directly**. All code changes related to your task should be committed to your isolated branch (`tool-a-attempt` for Tool A and `tool-b-attempt` for Tool B) before concluding the session.

Export AI Interaction Logs:

Follow the specific instructions in Section 5 to export chat conversations and retrieve request statistics. Save these as `tool_a_logs.zip`.

Clean Up and Ensure Isolation:

Return to main branch and remove the solution branch:

```
git checkout main
git branch -D tool-a-attempt
```

Important Branch Isolation Note

At this step, you will remove all changes made during the Tool A session from your local repository. Ensure that you have successfully created the patch file and exported all logs before proceeding, as these changes will no longer be recoverable.

Complete Tool A Survey:

Fill out the brief Tool A post-session survey.

Session 1 Checklist:

Before moving to Session 2, ensure you have completed the following:

- Created the patch file `tool_a_solution.patch` and deleted the `tool-a-attempt` branch.
- Collected AI interaction logs and packaged them as `tool_a_logs.zip`:
  - Exported all chat conversations and packaged them as `tool_a_chat_conversations.zip`.
  - Captured Copilot request usage screenshots as `tool_a_copilot_requests_start/end.png`.
- Completed the Tool A post-session survey.

4.3. Session 2: Tool B

Follow the same workflow as Session 1, but with the following adjustments:

- Create a branch named `tool-b-attempt` from the clean `main` branch.
- Use **Tool B** following its setup instructions in Section 3.
- After the 3 hour mark or upon task completion, generate the patch file as `tool_b_solution.patch`:

```
git diff main tool-b-attempt > tool_b_solution.patch
```

- Export logs as `tool_b_logs.zip` following the same procedure as Tool A.
- Complete the Tool B post-session survey.

> **Important Session 2 Note**
>
> You may start Session 2 immediately after Session 1 or take a break. Ensure you are ready to approach Session 2 from scratch, maintaining a fresh perspective and minimizing any influence from Session 1.
>
> To accurately collect all data from Session 2, please **make absolutely sure you capture both start and end Copilot usage screenshots for this session and that you have all the artifacts from Section 4.2**.

Session 2 Checklist:

Before concluding your work, ensure you have completed the following:

- Created the patch file `tool_b_solution.patch` and deleted the `tool-b-attempt` branch.
- Collected AI interaction logs and packaged them as `tool_b_logs.zip`:
  - Exported all chat conversations and packaged them as `tool_b_chat_conversations_json.zip`.
  - Captured Copilot request usage screenshots as `tool_b_copilot_requests_start.png` and `tool_b_copilot_requests_end.png`.
- Completed the Tool B post-session survey.

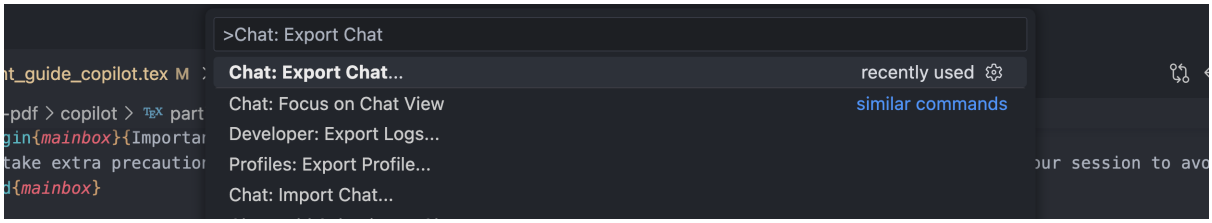
5. Exporting AI Interaction Logs from VS Code (Copilot)

To export your AI interaction logs (chat conversations and AI request statistics) from Copilot, follow these steps for both sessions.

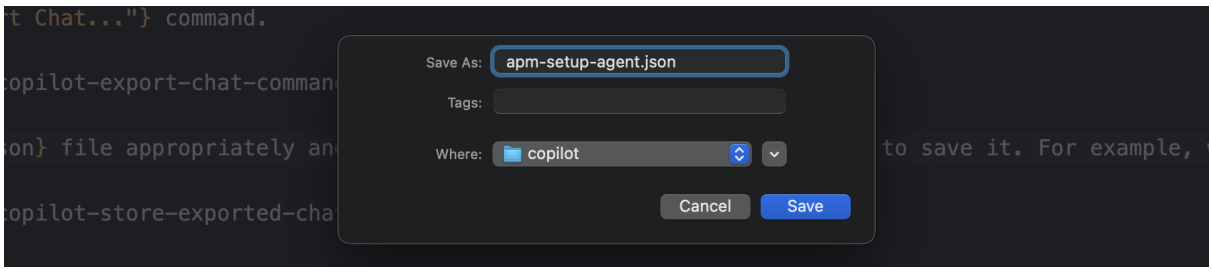
5.1. Export Chat Conversations

When using Tools A or B, you may open multiple chat conversations (e.g., at least three with APM: Setup, Manager, and Implementation Agents; one or two with Spec-kit or Plan agent). **Retain all chat conversations for our analysis. Do not delete any during your sessions**. To export chats, follow these steps:

- For each chat conversation opened during your session, **navigate to that specific chat** in the VS Code sidebar so it is your **active, visible chat tab**.
- Open the Command Palette (`Cmd+Shift+P` on macOS or `Ctrl+Shift+P` on Windows/Linux).
- Type and run the **"Chat: Export Chat..."** command.



- Name the exported `.json` file appropriately (e.g., use the agent name for APM conversations) and choose a location on your local machine to save.



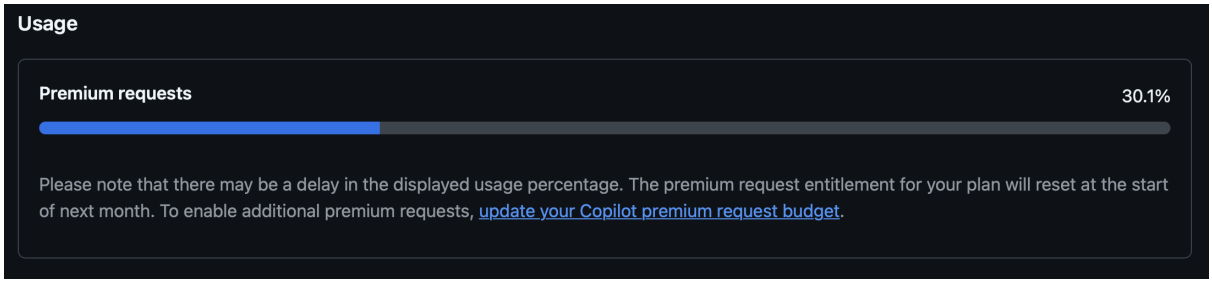
Repeat this process for all conversations used during the session. Once all conversations are exported, compress them into a single archive named `tool_a_chat_conversations_json.zip` for Tool A and `tool_b_chat_conversations_json.zip` for Tool B.

> **Important Chat Export Note**
>
> To take extra precautions, consider exporting conversations **immediately after completion** your session to avoid accidental deletions or technical issues. For example, in APM the Setup Agent conversation can be exported right after the Setup Phase is complete.

5.2. Retrieve AI Request Usage

Copilot does not provide a detailed, time-stamped CSV of token usage, cost and other metrics per request. Instead, we will capture your total request usage through Copilot's Settings.

- At the **start and end of your session**, open GitHub's Copilot Settings.
- In the **Usage** section, take a screenshot of the **Premium Requests percentage bar**. The difference between the start and end screenshots will be used to estimate your Copilot requests for the session.



- Save the screenshots using proper naming: `tool_a_copilot_requests_start.png` and `tool_a_copilot_requests_end.png` for Tool A, and `tool_b_copilot_requests_start.png` and `tool_b_copilot_requests_end.png` for Tool B.

> **Important Note on Request Metrics**
>
> We acknowledge this metric is less granular. The primary data for this study will come from the detailed **JSON chat exports**, which allow us to analyze the interaction patterns. The request count provides a secondary, high-level overview.

5.3. Package Logs for Submission

Finally, compress the exported chat conversations and your request count screenshot into a single archive for submission. For example:

```
zip -r tool_a_logs.zip tool_a_chat_conversations_json.zip
    tool_a_copilot_requests_start.png tool_a_copilot_requests_end.png
```

5.4. Log Archives Summary

After completing the above steps for both sessions, you will have two log archives:

- `tool_a_logs.zip` for Tool A
- `tool_b_logs.zip` for Tool B

These will be submitted along with your code patches in the final submission step.

6. Final Submission

After completing both sessions, submit all your data to our anonymous collection form.

Submission Checklist

Please be sure to upload the following files:

```
tool_a_solution.patch          # Code patch from Tool A session
tool_b_solution.patch          # Code patch from Tool B session
tool_a_logs.zip                # AI interaction logs from Tool A session
tool_b_logs.zip                # AI interaction logs from Tool B session
```

7. Contact & Help

This study is conducted for a Bachelor's thesis by Kostantinos Chaidemenos - CobuterMan.

If you encounter any technical issues or have questions about the study procedure, contact:

```
sdi2200262@di.uoa.gr        # Email
cobuter_man                 # Discord
```

Thank you again for your time and contribution to this research.

